

The *Clone* Strikes Back: Efficient Vulnerable Code Detection in Custom Android-based Systems

Esteban Luques*, Carlo Mazzocca⁺, Güliz Seray Tuncay[§], and Selcuk Uluagac*

*Cyber-Physical Systems Security Lab, Florida International University, Miami, Florida, USA

⁺University of Salerno, Fisciano, Italy

[§]Google, San Francisco, CA, USA

{eluqu004, suluagac}@fiu.edu, cmazzocca@unisa.it, gulizseray@google.com

Abstract—Open-source operating systems now power a wide range of devices, from wearables and smart appliances to vehicles. To meet specific functional, security, privacy, or hardware compatibility requirements, developers often customize these systems. However, such changes often carry a cost: downstream maintainers may overlook critical security patches or inadvertently reintroduce previously mitigated vulnerabilities when integrating new features and legacy drivers. In this study, we present a novel framework designed to efficiently detect vulnerable code in Custom Operating Systems (COSs) through a dual-layer architecture that identifies both insecure code clones and the absence of expected security-critical patches. While applicable to any COS, we evaluate our framework on Android and refer to it as ANDROVET. Being one of the world’s largest software platforms, Android exhibits systematic fragmentation in security fixes due to its diverse vendor variants and heterogeneous patching practices, making it an ideal use case scenario. We implemented and open-sourced our framework, and thoroughly evaluated it on 14 real-world Android-based COSs. ANDROVET detected 332 vulnerability instances (corresponding to 94 unique CVEs) across 14 COSs, and a GrapheneOS case study confirmed 15 absent patches from that distribution. Experimental results show that our framework achieves higher vulnerability-detection accuracy while reducing analysis time, running on average 4 times faster than existing tools.

Index Terms—Vulnerability Analysis, Code Clones, Custom Operating Systems, Patch Verification

1. Introduction

The modern software ecosystem relies heavily on large-scale open-source Operating Systems (OSs). From smartphones and embedded systems to automotive platforms, developers frequently fork upstream codebases to meet specific functional requirements, enhance privacy, or support legacy hardware, producing what we refer to as Custom Operating Systems (COSs). Notable examples span multiple domains: in the mobile space, LineageOS [1] (~881,000 active users), dotOS [2] (~755,843 downloads), and GrapheneOS [3] (~250,000 users) represent widely adopted Android-based COS; in the networking and embedded context, OpenWrt [4], [5] powers millions of embedded devices, including routers and gateways; and in the automotive domain, Automotive Grade

Linux [6] exemplifies industry-scale OS customization for vehicular platforms.

While COSs foster rapid innovation, extended device longevity, and software reusability, they introduce a significant security challenge: as upstream projects release critical patches, COS maintainers must manually backport or adapt these fixes into heavily modified codebases, a process prone to silent regressions where patches are overlooked, partially implemented, or inadvertently broken by deep source code modifications and the integration of legacy drivers. The difficulty is compounded by the extent to which COS diverges from its upstream ancestors. Security patches may undergo significant changes during integration, rendering traditional signature-matching and clone-detection tools ineffective. Moreover, such tools cannot detect the absence of an expected patch, nor reason about partial matches. For instance, they are unable to determine how many relevant code fragments must be present or absent, and in which location, for a CVE to be considered effectively mitigated. Similarly, community-driven projects are particularly susceptible to faulty or malicious patches, further underscoring the need for robust mechanisms capable of efficiently detecting both vulnerabilities and their expected fixes.

Limitations of Existing Solutions. Existing solutions typically rely on code clone detection to uncover vulnerabilities in source code [7]–[19]. While effective at identifying syntactic or structural similarities, these methods struggle to detect vulnerabilities caused by the absence of expected code fragments or incomplete patching. For example, a vulnerability caused by missing input validation may be fixed by adding a bounds check before processing user input. Such a patch may go undetected, as clone-based approaches focus on the presence or removal of vulnerable fragments rather than newly introduced checks.

Depending on the underlying technique, existing approaches exhibit different trade-offs. Hash-based approaches [8], [9], [20] are computationally efficient but brittle to code restructuring, while slicing and learning approaches [21]–[23] are semantically expressive but computationally expensive and language-dependent. For example, ReDebug [8] operates at the file or system level, often resulting in inefficiencies and false positives due to matches in unrelated files or functions, a problem that even worsens when patches have been modified. Another relevant framework is MOVERY [24]. Although it is designed to handle code transformations, its language sup-

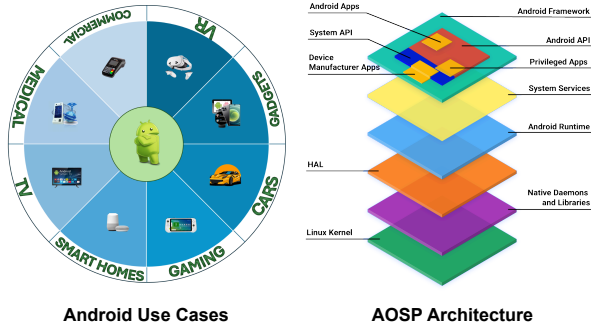


Figure 1. Overview of the main domains where Android is used and the layers composing its architecture.

port is limited, and may incorrectly associate semantically unrelated code fragments, generating spurious matches at high computational cost.

To overcome these challenges, we propose a novel dual-layer framework for source code analysis that combines token-based and structural encoding techniques to improve clone detection accuracy. Our solution goes beyond identifying vulnerable clones as it also detects the absence of an expected patch, a critical limitation overlooked by earlier works. The proposed abstraction technique leverages variable behavior patterns to effectively handle syntactic variations introduced by downstream customizations. Accordingly, given a target CVE, our framework first identifies all affected files and analyzes them to identify vulnerable code sections, then correlates these findings with the CVE reference information stored in the framework’s database. Subsequently, our framework computes a *found-to-expected* clone ratio, weighting each clone by its type (e.g., added or deleted lines) to estimate the likelihood that the analyzed system exhibits the vulnerability.

Android as a Representative Ecosystem. Throughout this paper, we apply our framework to the Android Open Source Project (AOSP) ecosystem, a representative and high-stakes environment, and refer to it as ANDROVET. Android is one of the world’s largest software ecosystems with hundreds of downstream variants and regular security-patch releases [25]. Its scale, combined with heterogeneous vendor practices, ranging from selective patch adoption to vendor-specific patch modifications [26], produces a systematic fragmentation of security fixes across codebases, making it a uniquely measurable environment for studying patch omission at scale.

Since its initial release in 2008, Android has evolved into a dominant platform powering billions of devices across smartphones, wearables, smart TVs, and vehicle infotainment systems [27]. Its open-source nature [28] fosters extensive customization and innovation, but also introduces significant security risks [29], [30]. Figure 1 illustrates the main domains where Android is deployed and the key layers composing the AOSP. To mitigate known vulnerabilities, Android receives regular security updates throughout its support lifecycle [31]–[33]. However, approximately 51.4% of active devices run unsupported AOSP versions¹, and prior research [33], [34] shows that

even supported devices sometimes miss timely updates, falling short of vendor commitment.

Contributions. The main contributions of this work can be summarized as follows:

- We propose a novel dual-layer framework for detecting vulnerable code and the absence of security-critical patches in COSs, and open-source our implementation at: <https://github.com/Demeter2025/AndroVET>.
- We formalize a new code representation that encodes, for each instruction, its type, local usage patterns, relative control-flow level, and variable-usage relations.
- We build and release two datasets focused on AOSP vulnerabilities, providing a valuable resource for the research community.
- We extensively evaluate our framework, ANDROVET, across 14 Android-based COSs, including a detailed stress-test analysis of GrapheneOS, a security and privacy focused COS. Our results demonstrate that our approach consistently outperforms state-of-the-art frameworks in both vulnerability detection accuracy and analysis time.

2. Background

This section provides the necessary background to contextualize our work. We first overview the AOSP-based COSs landscape and the customization objectives that drive their development, then we review the taxonomy of code clones that underpins our detection framework.

2.1. Android-based Custom Operating System

The AOSP customization process modifies the Android source code to meet specific functional, performance, security, or usability objectives. These changes often require in-depth adjustments to low-level components such as the kernel, Hardware Abstraction Layers (HALs), system services, and configuration frameworks. COSs like LineageOS, GrapheneOS, and CalyxOS [35] exemplify distinct paradigms of AOSP customization. LineageOS focuses on broad hardware compatibility and device longevity, particularly for unsupported devices. This entails integrating community-maintained drivers, porting proprietary vendor binaries, and adapting device trees and kernel configurations to ensure continued functionality on legacy hardware. GrapheneOS prioritizes security and privacy, implementing robust measures such as enhanced verified boot, strict SELinux policies, and advanced app isolation to minimize attack surfaces. It also introduces features like per-connection randomized MAC addresses and revocable network permissions [36], [37] for fine-grained connectivity control. CalyxOS aims to strike a balance between security and usability, offering a privacy-respecting environment without compromising user accessibility. The main objectives of AOSP-based COS development can be summarized as follows:

Performance Optimization: Developers improve efficiency by disabling redundant services, optimizing resource management, and leveraging power-saving mechanisms. For instance, GrapheneOS eliminates unnecessary background processes and applies exploit-hardening

1. At the time of this writing, only Android 12 and newer are officially maintained.

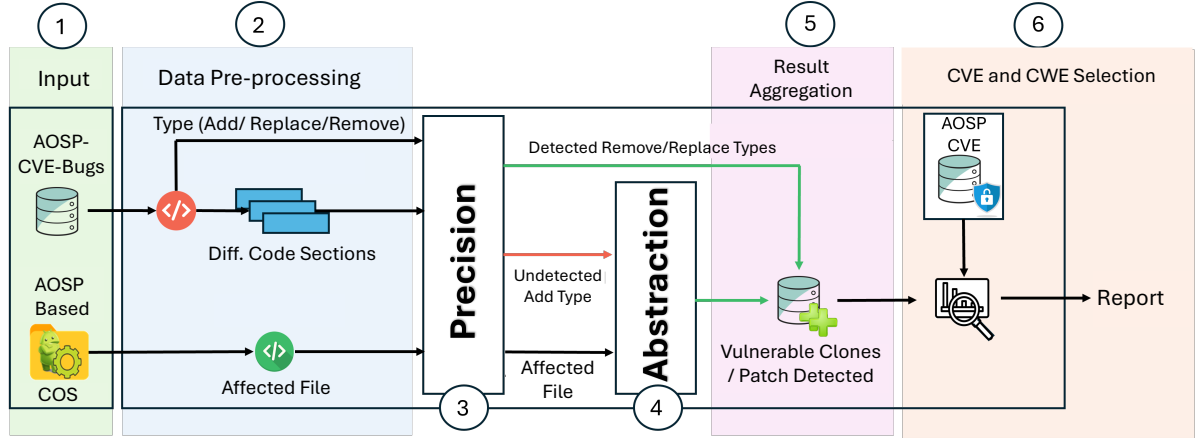


Figure 2. ANDROVET’s architecture and execution flow. The pipeline processes CVE data through six stages: (1) input of COS-CVE-Bugs and COS source files, (2) data pre-processing to classify change sites by type (i.e., add, remove, replace), split proposed change into code sections, and identify affected files, (3) Precision Layer detection of Type-1 and Type-2 clones, (4) Abstraction Layer detection of Type-2 and Type-3 clones for unmatched add-type changes, (5) results aggregation, computing the found-to-expected clone ratio per CVE, and (6) CVE and CWE selection producing the final vulnerability report.

techniques to reduce CPU and battery usage. CalyxOS enhances network handling to minimize superfluous data transfers, saving battery life.

- **Hardware Compatibility:** Since AOSP does not natively support a wide range of hardware, COS projects often integrate device-specific drivers and HALs to ensure operability. For example, LineageOS extends support to legacy devices through community-driven driver integration and kernel patching, enabling continued functionality beyond their official end-of-life.
- **Security & Privacy:** GrapheneOS strengthens security by introducing features such as memory tagging extensions (MTE) to mitigate vulnerabilities, a hardened malloc implementation for improved memory safety, and locked-down device encryption mechanisms. CalyxOS protects user data by replacing Google-dependent services with privacy-centric alternatives.
- **User Experience:** COS distributions frequently enhance UI usability. For instance, LineageOS supports extensive customization through theme engines, gesture-based navigation, and advanced developer settings, giving users granular control over the system interface.

2.2. Code Clones

Code clone detection is a fundamental technique in software engineering to identify redundant or duplicate code fragments within a code base. They are typically classified based on their degree of similarity and the nature of the changes between them. We adopt a widely accepted classification [38], which includes four main types:

- **Type-1:** Identical copies of code fragments, where the only difference may be white spaces, comments, and layout. These are the easiest to detect since there are no structural differences.
- **Type-2:** Code fragments have identical functionality, but they differ in terms of syntax, including variable

names, the order of statements, or the use of different but equivalent constructs.

- **Type-3:** Clones are semantically equivalent, but they may have more significant structural differences, such as different control flow structures or algorithms.
- **Type-4:** Code fragments are similar in functionality but implemented using different syntactic structures or styles.

3. ANDROVET

In this section, we present ANDROVET, a novel framework for detecting vulnerabilities in COSs. Our approach goes beyond the identification of code clones linked to known CVEs as it also verifies whether the corresponding patches have been properly applied. This dual capability is essential for minimizing false positives, as the presence of vulnerable code fragments alone does not confirm an active security risk, as mitigations may already be in place. Traditional clone-detection frameworks focus solely on locating vulnerable code, often neglecting cases where patches are missing. This oversight is significant, especially when patches consist of added instructions, such as input validation checks, rather than altering or removing existing code. We overcome these limitations by examining the contextual footprint of expected patches and detecting the absence of required lines. Figure 2 illustrates the architecture and execution flow of our framework, which is organized into two main layers:

- **Precision Layer:** This layer detects Type-1 and a subset of Type-2 code clones (specifically those resolvable through tokenization and rule-based matching). When code lines associated with security vulnerabilities are identified, it enforces an exact match (Type-1) to pinpoint potential security flaws with high accuracy. Conversely, when scanning for patch code, we tolerate minor modifications as long as they do not alter the program’s logic or behavior. If a match is not found in this layer, we proceed to the Abstraction Layer to search for more complex patches.

- **Abstraction Layer:** This layer detects Type-2 and Type-3 patch clones by identifying clone candidates (i.e., code segments that exhibit structural similarity to the enforced patch within the targeted files). Then, it flattens the code and analyzes its control flow and variable usage sequences. This generates a unique representation encoding the fundamental properties of the provided code.

The final decision on the likelihood of a vulnerability affecting or not the target system is made at the end of the results aggregation (see step 5 in Figure 2). In the following, we provide a detailed description of the layers that make up our framework and the operations performed to detect vulnerable code within a given COS.

3.1. Datasets

Our framework relies on two datasets: the first, named *OS-CVE*, contains known CVE records linked to the target standard distribution; the second, *OS-CVE-Bugs*, is derived from the former and includes detailed technical information on bugs associated with those vulnerabilities, including their patches. Both datasets used for evaluating ANDROVET are publicly available at our repository². To construct the *AOSP-CVE* (i.e., *OS-CVE*) dataset, given the focus of our work, we performed a systematic search across four widely used vulnerability databases (i.e., NVD, CVEDetails, MITRE, and the Android Security Bulletin) by filtering for entries that reference the term “Android.” This process yielded 29,168 records, which, after deduplication, resulted in 10,128 unique Android-related CVEs.

To build the *AOSP-CVE-Bugs* (i.e., *OS-CVE-Bugs*), we retrieved the corresponding diff code for each CVE when available, ultimately collecting diffs for 3,235 bugs across 3,150 CVEs. It is worth noting that a security vulnerability can refer to multiple distinct bugs. To support fine-grained analysis, each bug was decomposed into discrete change sites, i.e., specific code regions modified according to the corresponding diff files. The resulting dataset includes key metadata such as a unique ID, CVE-ID, BUG-ID, the index range of the change within the original COS file, the relative file path, and the associated repository folder. For each change block, we further split each change site into four components to enable a more comprehensive analysis (see Appendix B):

- **Cnt-bfr:** Context lines that appear before the change site.
- **Plus:** Lines that were added, or the updated versions of modified lines.
- **Minus:** Lines that were removed, or the original versions of modified lines.
- **Cnt-aft:** Context lines that appear after the change site.

The *AOSP-CVE-Bugs* dataset includes all CVEs published in the Android Security Bulletins, along with additional entries from NVD, MITRE, and CVEDetails [39] that pertain to AOSP modules. While the dataset is inherently constrained by the scope of publicly disclosed AOSP CVEs, we make no subjective selection choices and strive for comprehensive coverage.

2. <https://github.com/Demeter2025/AndroVET>

3.2. Code Pre-Processing

Each entry in our *AOSP-CVE-Bugs* dataset is categorized according to the operations needed to address the vulnerability: removing, adding, or replacing lines of code. This classification guides ANDROVET’s behavior during the detection phase. Our framework employs different line-filtering policies for each layer, and each detection type, i.e. `remove`, `add`, and `replace`. For vulnerable clone detection, it filters out trivial lines such as blanks, comments, and single-character statements (e.g., braces `{}`) as these do not meaningfully contribute to code similarity. In contrast, the Precision Layer retains comment lines during patch validation since they often help localize the affected code region. Once the relevant region is identified, ANDROVET shifts focus to the actual instructions to determine whether the patch has been correctly applied. To accurately locate the affected file within the source tree, ANDROVET extracts key details from the diff code, including:

- **Repository’s directory (RD):** The directory within the repository’s structure.
- **Relative path (RP):** The file location inside the RD.

The RD and RP are combined to construct the absolute path and identify the file location within the repository’s source tree. However, in COSs, file paths may differ slightly, or certain files might have been deleted or replaced. To address these issues, ANDROVET employs the following strategies: if a file cannot be located using the absolute path, ANDROVET searches the repository’s RD for occurrences of the file name. If multiple files with the same name are found, ANDROVET applies disambiguation strategies to select the correct one. For hardware driver files, it evaluates all occurrences, as they may repeat across drivers. For other files, it calculates the similarity between each path and the one in the diff file to make an informed decision.

When ANDROVET must infer a file’s absolute path, it only addresses CVEs marked as `replace` or `remove` to minimize false positives. Otherwise, evaluating missing patch code without certainty will lead to incorrect results. While this limitation may also apply to deleted or replaced lines, the possibility of a code clone in the wrong file is significantly lower than that of a missing patch. Moreover, ANDROVET’s detection ratio metric mitigates its impact.

3.3. Precision Layer

The Precision Layer avoids code abstraction and focuses on detecting Type-1 clones within the target COS. If the clone is detected, ANDROVET searches for corresponding patch code by applying specific rules to identify a subset of Type-2 clones. Specifically, ANDROVET:

- **Classifies pointers/variables** by inspecting their declarations to determine whether a pointer has been converted into a stack variable, potentially making the patch (e.g., a `memset()` addition) unnecessary.
- **Segments lines or merge them** to establish a common base between the source code and the proposed diff block.
- **Replaces equivalent values** such as `return 0` and `return NULL` (in c files).

- **Evaluates parameter changes** in functions, loops, or conditionals individually. If all parameters are present, the lines are matched.

Whenever a `replace` or `remove` clone (i.e., code that was modified or deleted in the reference patch) is detected, ANDROVET marks it as found only if both of the following conditions are met:

1) **No patch code is found.** ANDROVET performs an additional scan of the entire file to look for Type-2 clones, ensuring the proposed patch is absent.

2) **No conditioned execution.** Based on our experience with COSs, some vulnerabilities are mitigated by wrapping the vulnerable code in a conditional statement. For example, if a vulnerability arises from calling an unsupported method on certain devices, it can be addressed either by removing the call and implementing an alternative solution or by executing it conditionally. ANDROVET ensures that no new conditional statements have been introduced, wrapping the vulnerable code.

Any vulnerability with `add` blocks for which ANDROVET could not find the proposed patch is processed by the Abstraction Layer for Type-2 and Type-3 detection.

3.4. Abstraction Layer

ANDROVET analyzes code changes presented as short diff snippets, a context-starved setting that poses significant challenges for semantic analysis techniques such as Program Dependence Graphs (PDGs) and Code Property Graphs (CPGs). These methods typically require complete, buildable programs to resolve data flow, variable types, and control structures. When only isolated code fragments are available, the lack of global context makes it technically infeasible to construct fully accurate PDGs or CPGs [40].

To address this challenge, we introduce the Similarity Vector (SV), a localized feature representation and similarity metric specifically designed for context-free code fragments. The Abstraction Layer leverages SVs to detect Type-2 and Type-3 clones of patch fragments across a variety of programming languages (See full example in section 4). Each fragment is transformed into a language-agnostic structural encoding that captures key attributes such as instruction types, control-flow hierarchy, and variable usage patterns.

For example, consider the CVE-2015-6598, shown in Listing 1. Since the code may have evolved across versions and COSs developers may have modified it in unpredictable ways, we construct a structural abstraction to capture its essential characteristics. We formally define this representation in the following:

Preliminaries and Notation. Let $\mathcal{L} = \{L_1, L_2, \dots, L_m\}$ denote a contiguous set of source code lines. Given $\mathcal{L}$, we derive a normalized instruction sequence C , defined as follows:

$$C = (I_1, I_2, \dots, I_n), \quad (1)$$

where each I_i represents a *complete instruction* obtained through the following normalization process:

1) **Line aggregation:** Consecutive lines in $\mathcal{L}$ are merged as needed to yield syntactically complete instructions.

2) **Structural normalization:** Nested conditional constructs are flattened into a cohesive, semantically equivalent structure to ensure consistent representation across program versions.

3) **Assignment decomposition:** Compound or chained assignments (e.g., `var x = var y = 100`) are decomposed into separate single-variable assignments, each represented as an independent instruction.

The resulting fragment C represents a *canonicalized contiguous block of code*, stable under equivalent transformations applied during version comparisons. eq. 1 defines the final normalized representation used throughout our analysis. For a sequence C , we use tokenization and normalization to remove formatting and rename identifiers with placeholders (`VAR01`, `STR01`, etc.). For each instruction I_i we define:

- $T_i \in \mathcal{T} = \{A, F, C, L, R, D, P, I\}$
(Instruction types: Assignment (A), Function (F), Conditional (C), Loop (L), Return (R), Declaration (D), Print/Log (P), Other (I)).
- $p_i \in \mathbb{N}$: Nesting depth relative to the first instruction.
- For each variable placeholder u appearing in I_i :
 - $\text{class } X(u) \in \mathcal{X}$ where $\mathcal{X} = \{\text{VAR}, \text{STR}, \text{B}\}$,
 - $(c_i^{(t)}(u), k_i^{(t)}(u))$: the number of occurrences of u in instructions of type $t \in \mathcal{T}$ before and after I_i , respectively.
- Two section counters: a_i, b_i representing additional instances of T_i before and after position i .

Hence, each instruction maintains a small map of type-specific counters, e.g., if u appears once before an assignment A and twice after it in A and once in a subsequent conditional C , then $(c_i^{(A)}, k_i^{(A)}) = (1, 2)$ and $(c_i^{(C)}, k_i^{(C)}) = (0, 1)$.

Section Encoding and Similarity Vector. Each instruction is encoded as a *section* as follows:

$$s_i = (T_i, a_i, b_i) @ p_i((X_{i,1}, \{(t, c_{i,1}^{(t)}, k_{i,1}^{(t)})\}_{t \in \mathcal{T}}), \dots, (X_{i,m_i}, \{(t, c_{i,m_i}^{(t)}, k_{i,m_i}^{(t)})\}_{t \in \mathcal{T}})), \quad (2)$$

where the first tuple describes the instruction type and its local frequency context, and the second tuple enumerates its variable-class triples with per-type counters. The Similarity Vector of a contiguous fragment of source code C is given by the ordered concatenation:

$$SV(C) = s_1 :: s_2 :: \dots :: s_n. \quad (3)$$

We define two use cases: the *Reference SV* (RSV) from a patch fragment, and the *Source SV* (SSV) from the source code under analysis.

Derived-SSV Generation. Because repository code may include insertions or deletions, ANDROVET compares the RSV to derived subsequences of the SSV that preserve the same relative control-flow pattern. Let $R = (r_1, \dots, r_n)$ and $S = (s_1, \dots, s_m)$. A subsequence $D = (s_{i_1}, \dots, s_{i_n})$ of S is a valid *derived-SSV* of R if and only if the chosen indices are strictly increasing in S , that is

$$1 \leq i_1 < i_2 < \dots < i_n \leq m, \quad (4)$$

and it meets the following conditions:

$$T(s_{i_k}) = T(r_k), \quad \forall k = 1 \dots n, \quad (5)$$

where both R and D have the same instruction type sequence, and the sequence of nesting depth differences is preserved. That is:

$$(p_{r_{j+1}} - p_{r_j}) = (p_{i_{j+1}} - p_{i_j}), \quad \forall j = 0 \dots n-1, \quad (6)$$

To limit complexity, subsequences are drawn from a sliding window of width W around the context anchors (`cnt_bfr/cnt_aft`).

Per-Section Similarity. Given two sections r and d , the per-section similarity is given by:

$$\text{simSec}(r, d) = w_T \text{sim}_T(r, d) + w_V \text{sim}_V(r, d), \quad (7)$$

with $w_T + w_V = 1$ (defaults 0.25, 0.75). These values were selected through iterative sensitivity analysis and follow the proportional difference between the relationships that can be established between sections and between variables. Our analysis showed that variable relationships and contextual footprint contribute significantly more to accurate customized patch matching than instruction types, which guided the final configuration. Notably, the system remains stable across a broad parameter range making these values a robust operating point rather than a finely tuned optimum.

Instruction-type Similarity.

$$\text{sim}_T(r, d) = \begin{cases} 1, & T_r = T_d, a_i^{(r)} \leq a_i^{(d)}, b_i^{(r)} \leq b_i^{(d)} \\ \alpha, & T_r = T_d, a_i^{(r)} > a_i^{(d)} \text{ or } b_i^{(r)} > b_i^{(d)} \\ 0, & \text{otherwise,} \end{cases}$$

where $\alpha \in [0, 1)$ is a partial-match weight. The penalties for α derive from the counters a_i and b_i and follow the same rules as those in sim_V .

Variable-usage Similarity. Let V_r and V_d be the sets of variable placeholders in the RSV section r and the candidate SSV section d , respectively. Variables are treated as commutative inside a section, and each $u \in V_r$ seeks the best match among the unused $v \in V_d$ of the same identifier class X . Each SSV variable can be matched at most once and each RSV variable u contributes equally to the section's total variable budget, such that

$$\text{val}_u = \frac{1}{|V_r|}, \quad \sum_{u \in V_r} \text{val}_u = 1. \quad (8)$$

This value is divided evenly between the counters present in u , so for g counters:

$$\text{cnt}_i = \frac{1}{g} \text{val}_u = \frac{1}{g|V_r|}, \quad g > 0 \quad (9)$$

and the value of cnt_i is further divided evenly between their *historical* and *future* parts:

$$\text{half}_{\text{cnt}_i} = \frac{1}{2} \text{cnt}_i = \frac{1}{2g|V_r|}. \quad (10)$$

Each variable has one pair of counters per instruction type $t \in \mathcal{T}$, where $\mathcal{T} = \{A, F, C, L, R, D, P, I\}$. Thus, for u we denote

$$\mathbf{c}_r(u) = \{c_{r,t}(u) \mid t \in \mathcal{T}\}, \quad \mathbf{k}_r(u) = \{k_{r,t}(u) \mid t \in \mathcal{T}\},$$

and similarly for v in the candidate section. Each $c_{r,t}(u)$ (resp. $k_{r,t}(u)$) counts the number of type- t instructions before (resp. after) u within the section. For each type t , and $\delta \in \{c, k\}$ the deficit is

$$\Delta_{\delta,t}(u, v) = \max(0, \delta_{r,t}(u) - \delta_{d,t}(v)) \quad (11)$$

Each missing reference (each unit of deficit) incurs a penalty of $\frac{1}{\pi} \text{half}_{\text{cnt}_i}$ where π is the total amount of expected relations for that counter. This allows a controlled degradation of the confidence in the variable u instead of a binary jump. Aggregating over instruction types gives the per-counter contributions:

$$\text{part}_{s_\delta}(u, v) = \max(0, \text{half}_{\text{cnt}_i} - \sum_{t \in \mathcal{T}} \frac{\Delta_{\delta,t}(u, v)}{\pi_\delta^{(t)}}) \quad (12)$$

Then, the total contribution of a matched pair (u, v) is:

$$s(u, v) = \sum_{\delta \in \{c, k\}} \text{part}_{s_\delta}(u, v), \quad 0 \leq s(u, v) \leq \text{val}_u \quad (13)$$

Each $u \in V_r$ is paired with the available $v \in V_d$ of the same class that maximizes $s(u, v)$; once paired, v is removed from the candidate set. If no eligible match exists, $s(u) = 0$. Finally, the section-level variable-usage similarity is defined as

$$\text{sim}_V(r, d) = \sum_{u \in V_r} s(u), \quad 0 \leq \text{sim}_V(r, d) \leq 1. \quad (14)$$

Conditionals Encoding. Conditional instructions are a special case in SV abstraction and are represented by their conditions only, since their relative nesting is already captured by the nested depth level p_i . For a conditional instruction I_i of type C (e.g., `if (cond_1 && cond_2) { ... }`), we encode its conditions as an *unordered set* of atomic conditions:

$$H_i = \{h_{i,1}, h_{i,2}, \dots, h_{i,q}\}. \quad (15)$$

Each atomic condition $h_{i,j}$ is encoded as a section using the same structure as eq. 2 without the main section counters and p_i value:

$$h_{i,j} = (T_j^{(h)}) ((X_{i,1}, \{(t, c_{i,1}^{(t)}, k_{i,1}^{(t)})\}_{t \in \mathcal{T}}), \dots, (X_{i,m_i}, \{(t, c_{i,m_i}^{(t)}, k_{i,m_i}^{(t)})\}_{t \in \mathcal{T}})), \quad (16)$$

where $T_j^{(h)} \in \{\text{CMP}, \text{EQ}, \text{BOOL}\}$ is the condition type, and variable counters $(c_i^{(t)}, k_i^{(t)})$ are computed over the entire SV. The conditional section itself is defined as:

$$s_i = (C, a_i, b_i) @ p_i(H_i) \quad (17)$$

Matching of Conditionals. The individual conditions within C are treated as *commutative*, allowing the order vary across different versions of the code. Let H^R and H^S denote the sets of atomic conditions for a pair of aligned conditional sections. The similarity of their conditions is computed as the best average pairing between conditions, defined by

$$\text{sim}_H(H^R, H^S) = \frac{1}{|H^R|} \sum_{r \in H^R} \max_{s \in H^S} \text{simSec}(r, s), \quad (18)$$

TABLE 1. PERFORMANCE METRICS ACROSS DETECTION THRESHOLDS. AT $\tau = 70$, THE CLASSIFIER ACHIEVES THE BEST BALANCE BETWEEN PRECISION (0.87) AND RECALL (0.85), YIELDING AN F1-SCORE OF 0.86

Threshold	Accuracy	Precision	Recall	Specificity	F1-Score
60	0.68	0.58	0.81	0.59	0.67
65	0.72	0.64	0.73	0.71	0.68
68	0.73	0.67	0.71	0.75	0.69
70	0.82	0.87	0.85	0.78	0.86
75	0.67	0.84	0.59	0.81	0.69
80	0.62	0.85	0.49	0.86	0.62
85	0.47	0.76	0.24	0.87	0.37
90	0.43	0.70	0.18	0.87	0.29
95	0.43	0.70	0.18	0.87	0.29

where $\text{simSec}(r, s)$ is the standard per-section similarity from eq. 7. This formulation ensures that logically equivalent headers differing only in the order of conditions (e.g., A & B vs. B & A) produce identical similarity scores. The conditional section similarity becomes

$$\text{simSec}(r, d) = w_T \text{sim}_T(r, d) + w_H \text{sim}_H(H^R, H^S) \quad (19)$$

with $w_T + w_H = 1$ (default $w_H = 0.75$).

Aggregate Similarity and Decision Rule. For aligned RSV $R = (r_1, \dots, r_n)$ and derived-SSV $D = (d_1, \dots, d_n)$ their total similarity is:

$$\text{Sim}_{\text{tot}}(R, D) = \frac{1}{n} \sum_{i=1}^n \text{simSec}(r_i, d_i), \quad (20)$$

and the RSV is considered present if

$$\text{Sim}_{\text{tot}}(R, D) \geq \tau, \quad (21)$$

where τ is the similarity threshold. The value of τ is a user-configurable parameter, allowing for tuning the system’s sensitivity based on specific needs. For our evaluation, we used a default value of 0.7.

Sensitivity Analysis and Threshold Selection. To assess the robustness of our patch-detection classifier, we performed a sensitivity analysis across detection thresholds ranging from 60 to 95. Since the Abstraction Layer relies on annotated entries generated by the Precision Layer (i.e., enriched representations that guide the interpretation of a code snippet) we selected all change sites labeled as `add` in the *AOSP-CVE-Bugs* dataset to obtain a representative sample of ANDROVET’s behavior. We then processed these change sites through ANDROVET. In total, 285 patch snippets were identified as relevant to the target system and successfully activated the Abstraction Layer.

As shown in Table 1, lower thresholds (60–68) yield higher recall but degrade precision and specificity, reflecting a more permissive classification regime that increases false positives. In contrast, thresholds above 75 lead to a steep drop in both recall and F1-score, showing that the classifier becomes overly conservative and fails to detect a substantial portion of valid patches. At $\tau = 70$, the classifier attains the best overall balance, achieving a recall of 0.85, an accuracy of 0.82, and an F1-score of 0.86 (see Figure 3). This setting provides strong detection capability while maintaining moderate false-positive rates, an essential property for identifying modified code clones in short, abstraction-driven snippets.

Figure 4 further supports this choice: the Youden’s J statistic ($J = \text{Sensitivity} + \text{Specificity} - 1$) reaches its

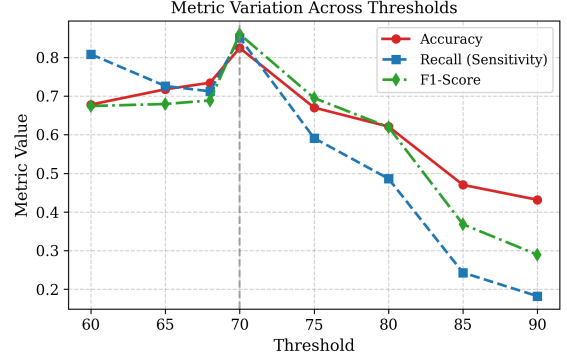


Figure 3. Accuracy, Recall, and F1-Score of the Abstraction Layer’s patch-detection classifier across similarity thresholds (τ). All three metrics peak at $\tau = 70$, confirming it as the optimal operating point.

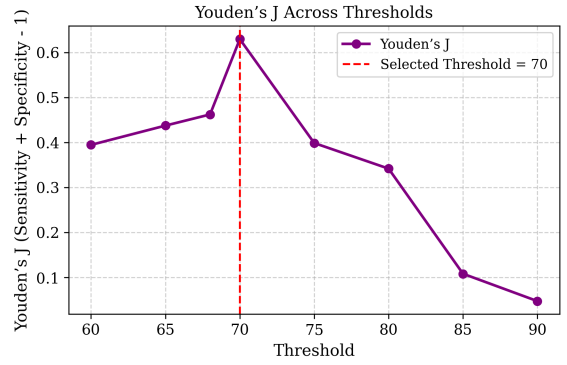


Figure 4. Youden’s J statistic across similarity thresholds (τ). The statistic reaches its maximum at $\tau = 70$, independently confirming the optimal threshold selected in Table 1.

maximum value ($J = 0.63$) at $\tau = 70$, confirming that this threshold yields the optimal trade-off between sensitivity and specificity. This balance ensures that the Abstraction Layer remains highly responsive to true modifications while robust against spurious matches. Accordingly, we select $\tau = 70$ as the deployment threshold as it maximizes statistical robustness while maintaining high precision and recall simultaneously.

Computational Costs of SVs. The construction of each SV, involves a sequence of preprocessing passes over the instruction stream, including tokenization, normalization, and identifier replacement, all linear in the number of instructions m . During the relationship encoding stage, every instruction and every variable are compared with all others to compute forward and backward reference counters and co-occurrence patterns, yielding a worst-case complexity of $O(n^2)$. The representation is language-agnostic and interpretable; each section corresponds to a specific instruction, so matches can be explained in terms of instruction and variable patterns. The matching function of ANDROVET also has a complexity of $O(n^2)$.

While the SV construction and matching algorithms have a theoretical worst-case complexity of $O(n^2)$ with respect to the number of instructions and variables n in the code fragment, because the Abstraction Layer operates on short diff snippets and source code segments, n is practically bounded to a very small constant. Because this quadratic operation is only performed on small size

```

namespace android{

SkipCutBuffer::SkipCutBuffer
(int32_t skip, int32_t cut){
+   if (skip < 0 || cut < 0 || cut > 64*1024){
+       ALOGW("out of range skip/cut: %d/%d,
+           using passthrough instead", skip, cut);
+       skip = 0;
+       cut = 0;
+   }
+   mFrontPadding = skip;
+   mBackPadding = cut;
+   mWriteHead = 0;
}
}

```

Listing 1. CVE-2015-6598 Diff code.

inputs, its cost is negligible in practice. Furthermore, the Abstraction Layer is only invoked selectively for add type changes that the Precision Layer fails to match. The majority of code is processed by the Precision Layer, which operates with linear complexity. This dual-architecture results in a near-linear system-level scalability demonstrated in our runtime evaluation (see Figure 7).

4. SV Creation Process - Full Example

To clarify the operation of the Abstraction Layer, this section presents a detailed example that also incorporates the SSV comparison rules. A complete, line-by-line illustration is provided in Figures 5–6 and Listing 1.

4.1. RSV creation

The RSV creation pipeline converts a code fragment into a structured, flow-preserving representation used by the abstraction layer. The main steps are:

- 1) **Instruction segmentation and normalization.** Tokenize and normalize the source (remove formatting, normalize literals, replace identifiers by placeholders such as VAR01, STR01) according to eq. 1.
- 2) **Nesting and type annotation.** As specified in eq. 2, for each instruction I_i record its type $T_i \in \mathcal{T}$, its relative nesting depth p_i , and compute the simple section counters a_i, b_i .
- 3) **Per-class variable counters.** For every variable placeholder u appearing at I_i , we compute the two scalar counters $c_i(u), k_i(u)$ (counts of occurrences of the same placeholder before and after I_i within the fragment). These counters are per-placeholder class (e.g., VAR, STR, B).

4.2. Deriving and Comparing SSVs

For each window W identified as candidate change site, ANDROVET performs the following operations:

- 1) Produce an SSV (following the RSV creation process), and crop it around the context anchors using the stored context to reduce noise and comparison cost
- 2) Generate monotone derived SSVs $D = (s_{i_1}, \dots, s_{i_n})$ that preserve instruction types and nesting-difference patterns and satisfy $1 \leq i_1 < \dots < i_n \leq m$ (see eq. 4).

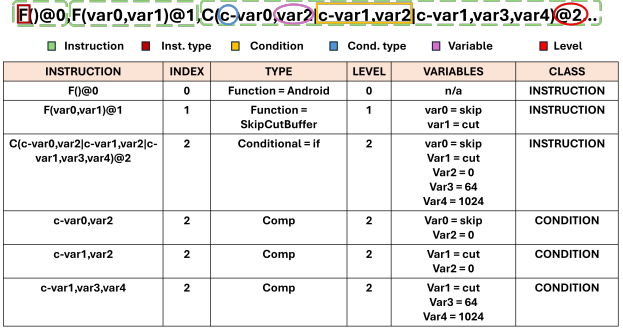


Figure 5. Intermediate Similarity Vector representation.

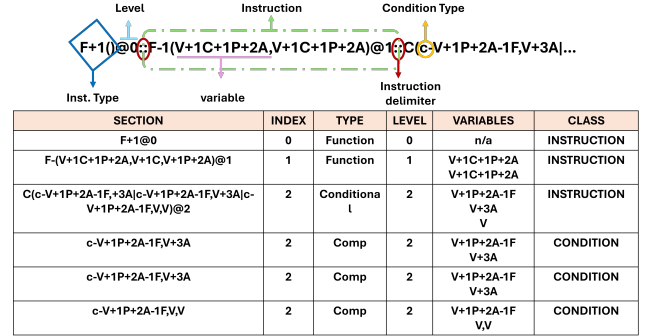


Figure 6. Similarity Vector representation.

- 3) Remove duplicates/subsets from the derived set and feed the remaining candidates to the comparison function. The best score per change-site is kept as the representative similarity value.

The following comparison rules summarize the scoring policy used by the comparison function. Let R be the RSV and S' a derived SSV. We compute section-level similarity in two steps. First, we compare the section counters of r_i against s_i . Sections are compared only when their instruction types are compatible. If the instruction type counts a_i, b_i differ, the section is penalized by following the same logic as for variable counters (see eq. 13).

Then, we match each variable $u \in V_r$ to an unused candidate variable $v \in V_{s'}$ of the same class in s_i and aggregate per-variable scores as in eq. 14. To implement this, we apply the following rules:

- 1) **One-time use.** Each SSV section is restricted by its positional index. Variables in each section are commutative and may be used at most once in the local matching procedure. Once assigned, it is marked as used and excluded from further pairing.
- 2) **Variable count precondition.** If a candidate section has strictly fewer variables of a required class than the RSV section, the variables portion of the section score is reduced: we mark the section as *variable-deficient* and set the maximum variable contribution to zero; only instruction-type counters remain eligible for scoring.
- 3) **Per-variable scoring and deficits.** For a matched variable pair (u, v) , we compute the per-counter deficits and penalize each missing reference by $\sum_{t \in \mathcal{T}} \frac{\Delta_{s,t}(u,v)}{\pi_s^{(t)}}$; counter scores are truncated at zero (see eq. 12 and eq. 13).
- 4) **Conditionals.** Conditional instructions are decomposed

TABLE 2. FEATURE OF ANDROID-BASED CUSTOM OPERATING SYSTEMS.

Project	Repo	Customization	Security & Privacy	UX/Design	Legacy Support
ArrowOS	ESP	✓	✗	✗	✓
AOKP	ESP	✗	✗	✓	✗
CalyxOS	ESP	✗	✗	✓	✗
crDroid	ESP	✓	✗	✗	✓
dotOS	ESP	✓	✗	✗	✓
Evolution-X	ESP	✓	✗	✗	✓
GrapheneOS	ESP	✗	✓	✗	✗
LineageOS	ESP	✓	✗	✗	✓
Paranoid Android	ESP	✓	✗	✗	✗
Replicant	ESP	✗	✓	✗	✗
Resurrection Remix OS	ESP	✓	✗	✗	✓

into their conditions; each condition is treated as a nested section and matched independently (the commutative matching is described in Section 3.4).

5) Fallback matching. If no variable counters exist for a section (e.g., template or namespace lines), a match is accepted when the instruction type and the number of placeholders are compatible.

Figure 5 shows the first step of the SV process given the code in Listing 1. The VAR placeholders repeat, and there are no counters. Figure 6 shows the final shape of the SV ready for matching. These rules are deliberately conservative: they favor precision over recall by requiring that a candidate reproduces the key usage relationships of the RSV.

4.3. Result Aggregation and CVE Likelihood

During analysis, ANDROVET records every matched or missing change site and its relationship with BUG-IDs and CVE-IDs. Let a vulnerability v be composed of N_v change sites, each with an expected modification type (addition, removal, or replacement). If the analysis detects F_v of these sites in a target repository, the *coverage ratio* is

$$\rho_v = \frac{F_v}{N_v}, \quad 0 \leq \rho_v \leq 1. \quad (22)$$

To help users prioritize their analysis, ANDROVET flags a CVE as a high-confidence match when $\rho_v \geq 0.75$ (see Figure 2, steps 5 and 6). This threshold reflects the fact that many vulnerabilities require numerous coordinated code changes, and cases where only a very small fraction of these changes appear (e.g., one match out of dozens) are highly unlikely to indicate a true vulnerability. The 75% cutoff therefore reduces false positives while maintaining confidence in identifying incomplete or partially applied patches, an aspect often overlooked by existing clone-based tools.

It is worth highlighting that this threshold is used exclusively for ranking and recommendation purposes within one of the ANDROVET reports. It serves purely as a heuristic to make ANDROVET final recommendations more robust against the small error margins inherent in abstraction-based clone detection. If the system were to require a perfect clone count match across all detectors, even a single misclassification could result in a false negative at the CVE level. The full, unfiltered list of all detected code clones for every CVE is always included in the detailed output, ensuring that no potential match is omitted.

5. ANDROVET Evaluation

This section presents a comprehensive evaluation of ANDROVET, comparing its performance against two popular, well-established tools [8], [9] and one state-of-the-art vulnerability detection framework [24] from the literature. Specifically, we assess each tool’s accuracy in identifying known Android vulnerabilities across 14 Android-based COS, as well as their execution time. Table 2 provides an overview of the selected projects and outlines their primary development objectives. Appendix A provides a detailed analysis for each COS, including discovered vulnerabilities, attack vectors, and the specific security properties compromised. We also perform a probabilistic analysis to highlight the most common CWE categories affecting these Android-based COS. All experiments were run on a server with an Intel Core i7 2.3 GHz (8 cores) and 32 GB RAM.

5.1. Baselines

ReDebug [8] is a well-established vulnerability detection tool that identifies known vulnerable code across large-scale software ecosystems. ReDebug requires the diff files for each CVE to be provided in a unified format. It parses these diffs to extract the original vulnerable code segments, which are then normalized (e.g., by removing whitespace and converting to lowercase). The normalized code is subsequently tokenized into sequences of n-grams to generate a unique vulnerability signature. To support fast matching, ReDebug pre-processes the entire target OS codebase by hashing the n-grams from each source file into a space-efficient Bloom filter database. During detection, it queries this database using the vulnerability signature to identify files that potentially contain unpatched clones of the vulnerable code. We selected ReDebug due to its practical applicability and its widespread adoption as a reference baseline in the literature.

UDDY [9] is a framework designed to detect vulnerable code clones in large-scale software projects. To use UDDY, users must generate an hmark file for the target codebase and upload it to UDDY’s web service to receive a vulnerability report. The tool offers two detection modes: Exact Matching and Abstract Matching. We evaluated both configurations, with the hmark generation process requiring approximately five hours per mode. As this framework was originally assessed on Android source code, it is one of the most relevant works in the literature for our comparison.

MOVERY [24] is the state-of-the-art vulnerability detection framework for identifying modified Vulnerable Code Clones (VCCs). Unlike traditional tools that rely on exact or near-exact clone detection, MOVERY addresses the syntax diversity introduced through internal updates and external modifications of OSS components. This is done by generating extensible and minimal vulnerability/patch signatures, leveraging core semantic lines, and applying selective abstraction and control/data flow analysis. We selected MOVERY as it shows high precision and recall in detecting vulnerabilities from heavily modified or reused OSS code. This framework uses both historical and disclosed vulnerable functions, offering a nuanced

TABLE 3. # OF DETECTED CVEs ACROSS ANDROID-BASED COS.

System	Version	AOSP	ANDROVET		ReDebug		VUDDY				MOVERY	
			#Android CVE	#CVE	#Android CVE	#CVE	Exact		Abstract		#Android CVE	#CVE
							#Android CVE	#CVE	#Android CVE	#CVE		
ArrowOS	13	13	16	16	60	60	15	162	17	147	1	84
AOKP	9	9	28	28	90	90	22	269	24	224	6	180
CalyxOS	N/A	14	20	20	45	45	✗	✗	✗	✗	✗	✗
crDroid	15	15	21	21	45	45	✗	✗	✗	✗	1	42
dotOS	dot6	12	25	25	93	93	✗	✗	✗	✗	1	98
Evolution-X	10.4	15	21	21	45	45	✗	✗	✗	✗	1	42
GrapheneOS	15	15	21	21	40	40	14	77	16	81	1	40
LineageOS	19.1	12	18	18	93	93	✗	✗	✗	✗	1	99
LineageOS	20.0	13	17	17	46	46	✗	✗	✗	✗	1	86
LineageOS	21.0	14	20	20	47	47	✗	✗	✗	✗	1	88
LineageOS	22.1	15	21	21	45	45	✗	✗	✗	✗	1	42
Paranoid Android	Topaz	13	21	21	42	42	✗	✗	✗	✗	1	42
Replicant ³	6	N/A	60	60	72	72	✗	✗	✗	✗	68	466
Resurrection Remix OS	8.6	10	22	22	79	79	19	236	20	212	4	138
Total			332	332	807	807	70	744	77	664	88	1447
Unique			94	94	180	180	25	70	27	342	71	606

✗= Results could not be obtained due to tool’s reliance on an external server or runtime failure (e.g. memory exhaustion). Total = Total CVE detection count over 14 Android-based COS. Unique = The actual CVE-ID count without repeated values. #Android-CVE = detected CVEs affecting AOSP based systems. #CVE = detected CVEs in total, including non-AOSP.

view of vulnerability propagation and making it a strong reference point for evaluating the robustness and accuracy of ANDROVET.

5.2. ANDROVET’s Detection Capability

Table 3 presents a comparison of ANDROVET’s results with those of ReDebug, VUDDY, and MOVERY. Notably, VUDDY consistently failed to analyze 10 out of the 14 available COS, limiting its overall effectiveness. ANDROVET successfully identified 332 vulnerabilities, corresponding to 94 unique CVEs. In comparison, VUDDY reported approximately 25 of these CVEs, while MOVERY detected 71. At first glance, ReDebug appears to be the most effective, reporting a total of 180 Android vulnerabilities. However, these results do not necessarily reflect superior detection capabilities. As discussed in the case study (see Section 6), the elevated counts reported by baseline tools largely result from inherent limitations that lead to substantial over-reporting. In particular, these frameworks exhibit a high false-positive rate: they fail to verify the presence of patches and often flag clones in incorrect or irrelevant files. Moreover, MOVERY is restricted to C/C++ files, missing a significant portion of Android CVEs, and produces a large number of redundant, low-impact clone matches.

In contrast, ANDROVET’s detection strategy is specifically designed to avoid the common pitfalls observed in tools like ReDebug and VUDDY. ANDROVET only flags a CVE as likely when both the vulnerable code is present and the corresponding patch is absent. This is achieved through a combination of precise matching in the Precision Layer and control-flow-aware abstraction in the Abstraction Layer. For instance, in the case of CVE-2019-2228, ReDebug incorrectly flagged a single clone in `http.c`, despite the patch involving multiple changes across several files. ANDROVET correctly identified all relevant modifications, accurately classifying the CVE as patched. Similarly, for CVE-2023-40093, ReDebug marked an unrelated file, whereas ANDROVET traced the actual change

sites and correctly concluded that the CVE had been addressed.

5.3. Runtime and Scalability Evaluation

To evaluate the runtime efficiency and scalability of ANDROVET, we benchmark its execution time against ReDebug and MOVERY across 14 custom Android ROMs. VUDDY is excluded from this comparison to ensure fairness, as it produced results for only 4 COS in our testbed. Including VUDDY with such limited coverage would risk introducing bias into the evaluation. A summary of the descriptive statistics for each tool’s performance is provided in Table 4.

Setup. To ensure a fair and consistent comparison, all tools were executed within a unified environment with identical hardware, OS settings, and input systems (COS). Because the baseline frameworks were originally designed with different assumptions about input formats and runtime environments, we applied a small set of non-invasive adjustments strictly limited to runtime compatibility. Importantly, none of these modifications impact the tools’ detection logic, matching heuristics, or internal analysis pipelines.

For ReDebug, the original tool terminates when encountering non-UTF-8 files. Since several Android COS contain mixed encodings, we added a lightweight wrapper that skips unreadable files while preserving ReDebug’s normal processing for all remaining inputs. This change only prevents premature termination and does not alter detection behavior.

For MOVERY, we observed that the tool relies on absolute paths and temporary files, and these are fragile when executed on large codebases. To allow MOVERY to complete its full pipeline on the larger COSs in our testbed, we introduced some environmental adaptations (e.g., replacing system-dependent paths with relative ones and ensuring temporary files do not conflict during repeated runs). These changes were limited to I/O handling and process management, leaving the vulnerability-

TABLE 4. DESCRIPTIVE STATISTICS OF EXECUTION TIMES PER TOOL (IN SECONDS).

	count	mean	std	min	25%	50%	75%	max	std_pct_mean	Overhead Execution vs ANDROVET
ANDROVET	14.0	4991.20	2493.05	1161.0	3950.75	4543.5	4949.5	10771.0	49.95	1.00×
ReDebug	14.0	21921.43	8157.71	5962.0	20348.50	24263.5	26430.00	36570.0	37.21	4.39×
MOVERY	13.0	18801.46	5553.32	12833.0	15368.00	18295.0	19242.00	35697.0	29.54	3.77×



Figure 7. ANDROVET consistently achieves the lowest runtime across all 14 Android-based COSs, with a mean of 4,991s compared to 21,921s for ReDebug and 18,801s for MOVERY

detection logic untouched. We intentionally avoided any modification that could influence its core analysis.

Results. As shown in Figure 7, ANDROVET consistently outperforms other frameworks, achieving the lowest mean execution time of 4,991.20 seconds and the lowest minimum at just 1,161.0 seconds on the Replicant ROM. Even in the worst-case scenario when analyzing Paranoid, ANDROVET takes 10,771 seconds, outperforming both ReDebug and MOVERY. Moreover, ANDROVET execution time follows closely the required change site workload for the target COS, showing a near-linear relationship between them. This demonstrates that ANDROVET is particularly well-suited for large-scale analysis, time-sensitive pipelines, or environments with limited computational resources. Although ANDROVET exhibits a higher standard deviation relative to its mean, it remains faster than all other tools across nearly all test cases. Figure 8 illustrates that even in outlier scenarios, ANDROVET absolute runtime stayed well below the worst-case values observed in ReDebug and MOVERY.

ReDebug shows the highest average runtime (i.e., 21,921 seconds) and the largest overall runtime on Paranoid (i.e., 36,570 seconds), making it the slowest and least scalable. MOVERY, with a mean runtime of 18,801 seconds, is still nearly three times slower than ANDROVET on average. We attribute this to several factors as MOVERY’s design includes a heavy pre-processing phase, which accounts for 85–90% of its total execution time. While this design improves runtime consistency as it achieves the lowest relative standard deviation, it also brings a significant overhead. Figure 8 further highlights ANDROVET’s advantage by showing that, despite its higher relative variance, it maintains consistently lower execution times across the full ROM set. In contrast, MOVERY and ReDebug suffer from both higher median runtimes and larger worst-case values. Therefore, ANDROVET demonstrates the best balance between speed and scalability, with consistently low execution times that

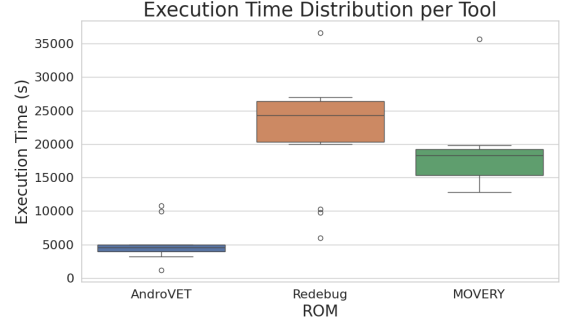


Figure 8. Despite higher variance, ANDROVET remains faster than both ReDebug and MOVERY in nearly all cases, with no outlier exceeding the median runtime of either baseline.

make it the most practical and efficient option for automated analysis across diverse Android ROMs. Although ReDebug and MOVERY offer distinct analytical features, their high runtime overheads significantly hinder their applicability in real-world, time-sensitive environments.

6. Case Study: GrapheneOS

In this section, we focus on GrapheneOS as a representative case study to enable a rigorous manual validation. Conducting an in-depth validation across all COS would require manually inspecting every matched or missing diff block, validating file locations, and reconstructing multiple CVEs, a process that scales to tens of thousands of clone candidates. This burden is compounded by the absence of authoritative ground truth: because Android-based COS diverge substantially from AOSP, no definitive record exists of which vulnerabilities are genuinely present or fully patched, making a comprehensive file-by-file audit the only reliable validation approach. Extending this effort to all 14 COS would therefore be disproportionate to its marginal scientific benefit.

We selected GrapheneOS [3] for in-depth validation on three grounds. First, its security- and privacy-focused design introduces substantial divergence from stock AOSP, making it a particularly challenging stress-test for patch-presence detection. Second, its active and transparent maintenance, with well-documented patch decisions, enables a rigorous and reproducible manual audit. Third, its wide adoption (~250,000 users) ensures that the findings carry practical relevance. Results for GrapheneOS are reported in Tables 3 and 6.

6.1. ANDROVET Evaluation

We evaluate ANDROVET on GrapheneOS using the *AOSP-CVE-Bugs* dataset. After filtering non-existent

files and trivial Type-1 matches (removes/replaces), ANDROVET identifies 3,236 candidate snippets. Following deduplication and removal of 27 comment-only and 227 whitespace-only changes, the final evaluation set consists of 2,538 change sites. As shown in Table 5, ANDROVET achieves high recall, 97.5% precision, and 96.82% F1-score, demonstrating its effectiveness in identifying unpatched security-relevant changes. For clarity, a positive case denotes a change site where ANDROVET should detect a clone (either an ‘added’ or ‘removed/replaced’ snippet). Conversely, a negative case denotes a change site where no such snippet should be found in the COS. Thus, the ratio between True Positive (TP) and True Negative (TN) reflects the ANDROVET correctness in identifying whether each expected code fragment is present or absent, independent of CVE-level conclusions.

TABLE 5. CONFUSION MATRIX FOR GRAPHENEOS.

Actual	Predicted	
	Positive	Negative
	Positive	Negative
	True Positive 2061 (81.7%)	False Negative 83 (3.3%)
	False Positive 52 (2.1%)	True Negative 334 (13.2%)

Since ANDROVET flagged 21 potential CVEs, we manually verified the results against the COS source code. Table 6 summarizes our findings. The ‘Patch Missing?’ column shows whether a patch is absent in the COS, while ‘ANDROVET Detection Correctness’ indicates whether our framework successfully identified its absence. To clarify the results in Table 6: a row such as *CVE-2015-6609* indicates that the clone(s) originate from an add type, the patch was absent in the COS source code, and ANDROVET correctly flagged the missing patch (✓), whereas other tools failed to do so. In contrast, red rows (e.g., *CVE-2019-2199*) correspond to false positives, where ANDROVET produced an incorrect conclusion (×). ANDROVET not only validated the presence of patches for CVEs that baseline tools failed to recognize, but also detected the absence of required security patches for 15 CVEs. In this challenging scenario, the baseline tools (ReDebug, VUDDY, MOVERY) failed to detect any potential CVE due to missing patch code, confirming that clone-detection approaches based solely on the presence of vulnerable code cannot identify security flaws caused by missing patch.

ANDROVET produced 6 false positives (marked with ×), flagging files as vulnerable where the patch was present. We believe that, in the context of COS security, a tool that proactively flags potential missing patches, requiring developer verification, is far more valuable than one that remains silent despite the presence of numerous security gaps. It is important to note that detecting a verified clone or the absence of a security-related instruction does not necessarily imply a vulnerability. ANDROVET is designed to highlight code sections that warrant review to ensure overall system security; since we work on COS and lack runtime information, we cannot claim the existence of an actual exploitable vulnerability.

TABLE 6. RESULTS ANALYSIS ON GRAPHENEOS.

CVE ID	Patch Type	Patch Missing?	AndroVET Detection Correctness	Baselines* Result
CVE-2015-6609	Add	Yes	✓	Miss
CVE-2015-6624	Add	No	×	—
CVE-2016-0804	Add	Yes	✓	Miss
CVE-2016-0831	Add	Yes	✓	Miss
CVE-2016-2420	Add	Yes	✓	Miss
CVE-2016-3875	Add	Partial	×	—
CVE-2016-3898	Add	Yes	✓	Miss
CVE-2017-0383	Add	Yes	✓	Miss
CVE-2019-2091	Add	Yes	✓	Miss
CVE-2017-7375	Add	Yes	✓	Miss
CVE-2017-0670	Add	Yes	✓	Miss
CVE-2017-0837	Add	Yes	✓	Miss
CVE-2019-2199	Add	No	×	—
CVE-2020-0239	Add	Yes	✓	Miss
CVE-2020-0248	Replace	No	×	—
CVE-2020-0386	Add	No	×	—
CVE-2020-0394	Add	Yes	✓	Miss
CVE-2021-0336	Add	Yes	✓	Miss
CVE-2021-0603	Add	No	×	—
CVE-2021-0645	Add	Yes	✓	Miss
CVE-2021-0646	Add	Yes	✓	Miss

*Baselines include ReDebug, MOVERY, and VUDDY.

×: False Positive (Code marked as missing but found present upon audit).

TABLE 7. CATEGORIZATION OF BASELINE TOOL LIMITATIONS ADDRESSED BY ANDROVET.

Limitation Category	Affected Tools
Missing patch verification	ReDebug, VUDDY
System-level analysis (wrong files)	ReDebug, VUDDY
Incomplete multi-site detection	ReDebug, VUDDY
Language/file type restrictions	MOVERY
Formatting/context insensitivity	ReDebug
High false positive rate	All baselines

6.2. Baseline Comparison and Key Limitations

We systematically compared ANDROVET against considered baselines, identifying critical weaknesses that ANDROVET explicitly addresses. Table 7 summarizes the main limitation categories.

ReDebug analyzed GrapheneOS in 7h 22m 49s, reporting 1,869 clone matches for 40 CVEs. We compared the tool’s output with the dataset of unified diff files and we observed that while we found the clones in the flagged locations, all these clones were unrelated (or at least not enough to represent a security concern) to the the marked CVEs. For example, CVE-2019-2228 and CVE-2023-40093, flagged based on only 1 change site for each one when they affect multiple files. Furthermore, the tool fails to capture the full extent of required changes; for instance, in CVE-2019-2228, ReDebug flagged a single clone in `/external/libcups/cups/http.c` despite the patch including multiple code modifications. The limitations of this tool stem primarily from ReDebug’s system-level analysis, which matches filenames while ignoring file paths (e.g., CVE-2015-6607 flagged 48 unrelated files). We identified five recurring patterns:

- 1) **Formatting-only patches:** Matches triggered by indentation changes.
- 2) **Conditional wrapping:** Vulnerable code wrapped in protective conditions.
- 3) **Wrong function context:** Clones in unrelated functions.

- 4) **Incomplete multi-site coverage:** Single matches for multi-location CVEs (e.g., CVE-2015-6607: flagged 48 unrelated files, while the CVE affects only 10).
- 5) **Bug/patch line overlap:** Misclassification of patched code.

In contrast, ANDROVET filters non-semantic changes, checks for conditional guards, leverages contextual correlation, and verifies overlapping patch content.

VUDDY (Abstract Matching) was evaluated using both abstract and exact matching. In the abstract configuration, it detected 459 clones corresponding to 81 CVEs. However, only 16 were Android-related, and of these, only 4 were located in the correct files, all involving critical issues:

- *CVE-2017-14491*: This CVE was marked based on a clone in `rfc2131.c`, but the vulnerability actually involves four different files.
- *CVE-2016-3834*: This CVE affects five different files. VUDDY reported five clones in `cameraSource.cpp`, which we manually checked and found to be correctly patched.
- *CVE-2017-0765*: VUDDY failed to recognize a type change (`size_t` to `ssize_t`) and marked a patched file as vulnerable.
- *CVE-2017-0814*: VUDDY identified 3 clones in `codebook.c`, but this CVE impacts 3 files with 30 distinct change sites. We manually checked and the CVE was patched.

The remaining AOSP-related CVEs were marked based on matches in the wrong files.

VUDDY (Exact Matching) found 537 clones for 77 CVEs. Of these, only 14 were Android-related, while 82% affected unrelated platforms (e.g., CVE-2019-9818 affects only Windows). Furthermore, matches frequently appeared in incorrect files. Manual verification confirmed that even where VUDDY identified the correct code segments, the corresponding patches were present, demonstrating the tool’s inability to distinguish between vulnerable and patched code.

These results highlight that VUDDY shares ReDebug’s limitations: incorrect file detection, lack of contextual awareness, and no verification (or at least not efficient) of patch presence. In contrast, ANDROVET overcomes these issues through: file-level analysis, selective abstraction applied only when precision demands it, and aggregation of clone counts relative to total change-site requirements.

MOVERY reported 40 distinct CVEs, but only one of them (CVE-2020-5235) was Android-related. Unfortunately, we could not verify the correctness of MOVERY’s detection of CVE-2020-5235 due to the absence of available diff code for that vulnerability. A broader limitation of MOVERY lies in its language support. The tool exclusively analyzes C and C++ source files. As a result, MOVERY is only able to attempt analysis on 1,343 out of 3,150 publicly known Android CVEs (approximately 42.6% of the total known set). This significantly restricts its applicability in modern Android security workflows. Additionally, MOVERY exhibited notable detection inefficiency, as illustrated by an edge case involving CVE-2020-15211 (a Windows Speech Runtime vulnerability). MOVERY reported 7,043 clone detections for this single

CVE, spread across 908 distinct functions. In contrast, the NVD record describes only 44 code changes across 30 C/C++ files. This highlights MOVERY’s tendency to produce redundant or low-impact clone reports, increasing the developer’s analysis workload. ANDROVET takes a more focused and context-aware approach, capable of detecting vulnerable code across all relevant file types. Its outputs are concise and more actionable, making it more practical for vulnerability analysis.

7. Discussion and Limitations

This section discusses the broader implications of our findings and examines the limitations of our approach. While the framework shows strong performance in detecting vulnerable code and missing patches across custom operating systems, several trade-offs and constraints remain. We outline these aspects, assess their practical impact, and suggest directions for future improvement.

Type-4. The SV is a structural encoding designed to operate on isolated diff snippets without requiring a complete, buildable program. As a result, our framework does not explicitly target Type-4 clones, where functionally equivalent behavior is achieved through entirely different structural constructs. Detecting such equivalences generally requires dynamic analysis or deep semantic reasoning, both beyond this work’s scope. However, in practice, this limitation is of limited impact: most security patches involve localized structural changes (e.g., added checks or refined conditions) rather than full semantic rewrites, making Type-4 clones rare in this setting. Semantically similar fragments may still be detected when sufficient structural similarity is preserved, though this is not guaranteed.

False Positives. Structural similarity may yield FPs in two identifiable cases: template-generated or macro-heavy code, where unrelated fragments share identical structural skeletons; and boilerplate regions such as error-handling or logging blocks, which frequently repeat the same instruction-type sequences across unrelated functions. In our work, both cases are mitigated by the aggregation mechanism across multiple change sites (see Eq. 22), which requires a high proportion of matched patch regions before flagging a vulnerability, thereby reducing the likelihood that coincidental structural matches lead to incorrect CVE-level conclusions. For example, this yields 97.5% precision in the GrapheneOS validation.

Behavioral Equivalence. Rather than proving behavioral equivalence, our goal is to detect the absence of known structural changes introduced by upstream security patches. Because our framework operates on isolated diff snippets, full control-flow analysis is infeasible; consequently, we do not guarantee behavioral equivalence between matched fragments.

Instead, the SV captures structural and usage-pattern equivalence through three invariants: (i) preservation of instruction-type sequences; (ii) preservation of relative control-flow nesting patterns; and (iii) compatibility of variable usage footprints across instruction types (see Eq. 2), which are sufficient to reflect security-relevant structural properties of patches.

Generalizability. While ANDROVET is validated on Android-based COS, the core components of our frame-

work are not inherently Android-specific. The SV abstraction and the dual-layer architecture operate on diff snippets and structural encodings that are language- and platform-agnostic in principle. With suitable parser support, path-disambiguation updates, and vulnerability datasets, the framework can be extended to other fork-based ecosystems, such as Linux kernel derivatives or other OS variants. We regard Android as a representative and demanding instantiation of a broader class of problems, rather than a limiting assumption of our design.

Evaluation. A significant challenge in this research domain remains the lack of authoritative ground truth within the COS ecosystem. Therefore, our in-depth validation is limited to GrapheneOS, as its security-focused customizations create conditions that effectively stress-test detection, while its active maintenance makes it a relevant and representative case study for real-world applicability.

Future Work. A promising direction for future work is a direct empirical comparison between ANDROVET and Vanir [41], [42], Google’s patch-detection framework. Such a comparison would provide additional insight into the trade-offs between signature-based and structural-abstraction approaches for patch verification.

8. Related Work

A wide range of clone detection techniques have been proposed, including direct text comparison [7], token-based approaches [8]–[11], [43]–[45], Abstract Syntax Tree (AST)-based methods [12], [13], [46]–[52], binary signature analysis [14]–[16], [53], and Machine Learning (ML) approaches [17]–[19], [54]–[56]. While each technique demonstrates effectiveness within its scope, it also exhibits inherent limitations, particularly when applied to vulnerability analysis in complex environments such as the AOSP. Several works have specifically focused on detecting vulnerable code clones. Among the earliest is ReDebug [8], which leverages tokenization to identify vulnerable code segments. VUDDY [9] is another notable framework, designed to detect vulnerable code clones in large-scale software projects. Seunghoon Woo et al. introduced MOVERY [24], a novel approach aimed at discovering vulnerable code clones through customized code detection. More recently, Google released Vanir [41], an open-source patch-detection tool that follows a signature-based methodology inspired in earlier approaches proposed by [8], [9]. Vanir is specifically designed to identify missing security patches in Android and Linux kernel codebases at scale.

However, existing approaches operate under varying assumptions that limit their effectiveness as comprehensive solutions. Most tools function either at the system or file level, often producing unrelated matches when analyzing the wrong file, or multiple matches across different functions within the same file, undermining their reliability. Furthermore, as patch code naturally evolves or is customized by developers, techniques focusing on Type-1 and Type-2 clones frequently produce false positives or miss modified patches entirely. While abstraction-based methods can tolerate greater variation, they introduce a critical trade-off: increased risk of incorrectly mapping

structurally similar but semantically unrelated code to known vulnerabilities.

Recent work has also explored the use of Large Language Models (LLMs) for vulnerability discovery. However, these methods operate as opaque black boxes, lacking the traceable evidence provided by ANDROVET. Although embedding-based models such as [57] are well suited for clone detection, they are trained on complete, well-formed functions rather than isolated diff snippets, and do not expose which specific property triggered the match. Furthermore, recent studies have highlighted significant accuracy limitations of LLMs in reliably identifying vulnerabilities [58], with performance degrading notably under semantic-preserving transformations [59]. These shortcomings render them ill-suited for compliance-oriented workflows, where a traceable and reproducible link between the analyzed code and the resulting verdict is essential. To the best of our knowledge, LLM-based approaches have not been applied to patch presence verification, leaving this critical aspect of vulnerability assessment unaddressed.

Fundamentally, the narrow focus of these tools on code clone detection, rather than the broader characteristics of vulnerabilities (e.g., affected files, multi-site changes, and contextual information), renders them inefficient for real-world vulnerability analysis.

ANDROVET overcomes these limitations by correlating multiple clones to a single underlying bug and detecting the absence of expected security-related instructions, which is a critical shortcoming observed across all existing detection methods in the literature.

9. Conclusion

We proposed a novel framework for detecting vulnerabilities in COSs that identifies code clones linked to known CVEs while also verifying whether the corresponding patches have been properly applied, a capability largely overlooked by existing approaches. Given the scale and complexity of the Android ecosystem, where widespread OS customization frequently introduces vulnerabilities that current analysis tools fail to detect, we validate our approach on 14 Android-based COSs and conduct a dedicated GrapheneOS case study. Experimental results demonstrate the effectiveness of our solution against well-established baselines.

Acknowledgment

We sincerely thank the anonymous reviewers as well as Hyunwook Baek, Camillus Cai, Mihai Christodorescu, and David Kleidermacher from Google for their helpful feedback on our paper and support for our work.

This work was partially supported via Florida International University Presidential Fellowship, US National Science Foundation’s Intergovernmental Personnel Act Independent Research & Development Program, Cyber Florida, and Google’s ASPIRE Program. The views expressed are those of the authors alone, not of the funding agencies. LLMs were used solely for grammar and clarity; all outputs were thoroughly reviewed and revised by the authors to ensure accuracy and integrity.

References

- [1] L. Project, "Lineageos stats," <https://stats.lineageos.org/>, 2023, [Online; accessed 27-Feb-2023].
- [2] dotOS Project, "dotos stats," <https://www.droidontime.com/stats>, 2023, [Online; accessed 27-Feb-2023].
- [3] Eirikr70 et al. (2024, March) Market share. GrapheneOS Discussion Forum, earliest post approximately March-2024. [Online]. Available: <https://discuss.grapheneos.org/d/1137-market-share>
- [4] J. Liu, Y. Shen, Y. Xu, H. Sun, H. Shi, and Y. Jiang, "Effectively sanitizing embedded operating systems," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3658490>
- [5] A. Roshandel and M. Egele, "Zeus-iot: Comprehensive code signing to prevent iot device weaponization," in *2025 IEEE Annual Computer Security Applications Conference (ACSAC)*, 2025, pp. 845–857.
- [6] S. Jeong, M. Ryu, H. Kang, and H. K. Kim, "Infotainment system matters: Understanding the impact and implications of in-vehicle infotainment system hacking with automotive grade linux," in *Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 201–212. [Online]. Available: <https://doi.org/10.1145/3577923.3583650>
- [7] C. K. Roy and J. R. Cordy, "Nicad: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization," in *2008 16th IEEE international conference on program comprehension*. IEEE, 2008, pp. 172–181.
- [8] J. Jang, A. Agrawal, and D. Brumley, "Redebug: finding unpatched code clones in entire os distributions," in *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012, pp. 48–62.
- [9] S. Kim, S. Woo, H. Lee, and H. Oh, "Vuddy: A scalable approach for vulnerable code clone discovery," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 595–614.
- [10] H. Sajnani, V. Saini, and C. Lopes, "A parallel and efficient approach to large scale clone detection," *Journal of Software: Evolution and Process*, vol. 27, no. 6, pp. 402–429, 2015. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/smr.1707>
- [11] M. A. Nishi and K. Damevski, "Scalable code clone detection and search based on adaptive prefix filtering," *Journal of Systems and Software*, vol. 137, pp. 130–142, 2018.
- [12] L. Jiang, G. Mishergahi, Z. Su, and S. Glondou, "Deckard: Scalable and accurate tree-based detection of code clones," in *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 2007, pp. 96–105.
- [13] Y. Xiao, B. Chen, C. Yu, Z. Xu, Z. Yuan, F. Li, B. Liu, Y. Liu, W. Huo, W. Zou, and W. Shi, "MVP: Detecting vulnerabilities using Patch-Enhanced vulnerability signatures," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1165–1182. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/xiao>
- [14] W. M. Khoo, A. Mycroft, and R. Anderson, "Rendezvous: A search engine for binary code," in *2013 10th Working Conference on Mining Software Repositories (MSR)*, 2013, pp. 329–338.
- [15] M. Bourquin, A. King, and E. Robbins, "Binslayer: accurate comparison of binary executables," in *Proceedings of the 2nd ACM SIGPLAN Program Protection and Reverse Engineering Workshop*, ser. PPREW '13. New York, NY, USA: Association for Computing Machinery, 2013. [Online]. Available: <https://doi.org/10.1145/2430553.2430557>
- [16] J. Pewny, F. Schuster, L. Bernhard, T. Holz, and C. Rossow, "Leveraging semantic signatures for bug search in binary programs," in *Proceedings of the 30th Annual Computer Security Applications Conference*, ser. ACSAC '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 406–415. [Online]. Available: <https://doi.org/10.1145/2664243.2664269>
- [17] M. White, M. Tufano, C. Vendome, and D. Poshyvanyk, "Deep learning code fragments for code clone detection," in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 87–98.
- [18] X. Gu, H. Zhang, and S. Kim, "Deep code search," in *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 933–944.
- [19] M. Tufano, C. Watson, G. Bavota, M. Di Penta, M. White, and D. Poshyvanyk, "Deep learning similarities from different representations of source code," in *Proceedings of the 15th international conference on mining software repositories*, 2018, pp. 542–553.
- [20] T. Kamiya, S. Kusumoto, and K. Inoue, "Ccfinder: A multilingual token-based code clone detection system for large scale source code," *IEEE transactions on software engineering*, vol. 28, no. 7, pp. 654–670, 2002.
- [21] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "Vuldeepecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2224–2236, 2019.
- [22] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.
- [23] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *2014 IEEE symposium on security and privacy*. IEEE, 2014, pp. 590–604.
- [24] S. Woo, H. Hong, E. Choi, and H. Lee, "MOVERY: A precise approach for modified vulnerable code clone discovery from modified Open-Source software components," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 3037–3053. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/woo>
- [25] R. Mayrhofer, J. Stoep, C. Brubaker, D. Hackborn, B. Bonné, G. S. Tuncay, R. Jover, and M. Specter, "The android platform security model (2023)," *arXiv*, 2023.
- [26] M. Mahmoudi and S. Nadi, "The android update problem: An empirical study," in *Proceedings of the 15th International Conference on Mining Software Repositories*, ser. MSR '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 220–230. [Online]. Available: <https://doi.org/10.1145/3196398.3196434>
- [27] Android. (2024) Android Open Source Project Documentation. [Online]. Available: <https://source.android.com/docs/devices>
- [28] Android Open Source Project. (2024) Android Open Source Project (AOSP). [Online]. Available: <http://source.android.com/>
- [29] Y. Aafer, X. Zhang, and W. Du, "Harvesting inconsistent security configurations in custom android {ROMs} via differential analysis," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 1153–1168.
- [30] X. Zhou, Y. Lee, N. Zhang, M. Naveed, and X. Wang, "The peril of fragmentation: Security hazards in android device driver customizations," in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 409–423.
- [31] K. R. Jones, T.-F. Yen, S. C. Sundaramurthy, and A. G. Bardas, "Deploying android security updates: an extensive study involving manufacturers, carriers, and end users," in *ACM Conference on Computer and Communications Security (CCS)*, 2020.
- [32] S. Farhang, M. B. Kirdan, A. Laszka, and J. Grossklags, "An empirical study of android security bulletins in different vendors," in *The Web Conference (WWW)*, 2020.
- [33] A. Acar, G. S. Tuncay, E. Luques, H. Oz, A. Aris, and S. Uluagac, "50 shades of support: A device-centric analysis of android security updates," in *Network and Distributed System Security (NDSS 24)*, 2024.
- [34] H. Deng, G. S. Tuncay, A. Acar, E. Luques, H. Oz, A. Aris, and S. Uluagac, "Unveiling the global landscape of android security updates," *IEEE Transactions on Dependable and Secure Computing*, 2026.

- [35] The Calyx Institute, “CalyxOS,” <https://calyxos.org/>, 2026, accessed: 2026-03-18.
- [36] Android, “Android permissions,” <https://developer.android.com/guide/topics/permissions/overview>, [Online; accessed 27-Apr-2026].
- [37] G. S. Tuncay, “Android permissions: Evolution, attacks, and best practices,” *IEEE Security & Privacy*, vol. 22, no. 6, pp. 40–49, 2024.
- [38] C. K. Roy, J. R. Cordy, and R. Koschke, “Comparison and evaluation of code clone detection techniques and tools: A qualitative approach,” *Science of Computer Programming*, vol. 74, no. 7, pp. 470–495, 2009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167642309000367>
- [39] CVEDetails, “Cvedetails website,” <https://www.cvedetails.com/>, 2023, [Online; accessed 27-Feb-2023].
- [40] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and discovering vulnerabilities with code property graphs,” in *2014 IEEE Symposium on Security and Privacy*, 2014, pp. 590–604.
- [41] H. Baek, D. Truong, J. Dunlap, L. Stan, and O. Chang, “Announcing the launch of vanir: Open-source security patch validation,” <https://security.googleblog.com/2024/12/announcing-launch-of-vanir-open-source.html>, December 2024, google Online Security Blog.
- [42] Google, “Vanir: Missing patch scanner,” <https://github.com/google/vanir>, 2024, GitHub repository.
- [43] Z. Li, S. Lu, S. Myagmar, and Y. Zhou, “Cp-miner: Finding copy-paste and related bugs in large-scale software code,” *IEEE Transactions on Software Engineering*, vol. 32, no. 3, pp. 176–192, 2006.
- [44] B. Hummel, E. Juergens, L. Heinemann, and M. Conradt, “Index-based code clone detection: incremental, distributed, scalable,” in *2010 IEEE International Conference on Software Maintenance*, 2010, pp. 1–9.
- [45] P. Wang, J. Svajlenko, Y. Wu, Y. Xu, and C. K. Roy, “Ccaligner: A token based large-gap clone detector,” in *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*, 2018, pp. 1066–1077.
- [46] Y. Yang, Z. Ren, X. Chen, and H. Jiang, “Structural function based code clone detection using a new hybrid technique,” in *2018 IEEE 42nd annual computer software and applications conference (COMPSAC)*, vol. 1. IEEE, 2018, pp. 286–291.
- [47] M. Chilowicz, E. Duris, and G. Roussel, “Syntax tree fingerprinting for source code similarity detection,” in *2009 IEEE 17th International Conference on Program Comprehension*, 2009, pp. 243–247.
- [48] L. Büch and A. Andrzejak, “Learning-based recursive aggregation of abstract syntax trees for code clone detection,” in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2019, pp. 95–104.
- [49] J. Li and M. D. Ernst, “Cbcd: Cloned buggy code detector,” in *2012 34th International Conference on Software Engineering (ICSE)*, 2012, pp. 310–320.
- [50] Y. Hu, D. Zou, J. Peng, Y. Wu, J. Shan, and H. Jin, “Treecen: Building tree graph for scalable semantic code clone detection,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE ’22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3551349.3556927>
- [51] R. Koschke, “Large-scale inter-system clone detection using suffix trees and hashing,” *Journal of Software: Evolution and Process*, vol. 26, no. 8, pp. 747–769, 2014. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/smr.1592>
- [52] Q. Zhao, C. Huang, and L. Dai, “Vuldeff: Vulnerability detection method based on function fingerprints and code differences,” *Knowledge-Based Systems*, vol. 260, p. 110139, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950705122012357>
- [53] H. Zhang and Z. Qian, “Precise and accurate patch presence test for binaries,” in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 887–902. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/zhang-hang>
- [54] N. Marastoni, R. Giacobazzi, and M. Dalla Preda, “A deep learning approach to program similarity,” in *Proceedings of the 1st international workshop on machine learning and software engineering in symbiosis*, 2018, pp. 26–35.
- [55] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, “A novel neural source code representation based on abstract syntax tree,” in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 783–794.
- [56] S. Karthik and B. Rajdeepa, “A collaborative method for code clone detection using a deep learning model,” *Advances in Engineering Software*, vol. 174, p. 103327, 2022.
- [57] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, “Codebert: A pre-trained model for programming and natural languages,” in *Findings of the association for computational linguistics: EMNLP 2020*, 2020, pp. 1536–1547.
- [58] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, “Vulnerability detection with code language models: How far are we?” *47th IEEE/ACM International Conference on Software Engineering (ICSE 2025)*, 2025.
- [59] N. Risse and M. Böhme, “Uncovering the limits of machine learning for automatic vulnerability detection,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 4247–4264. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/risse>

Data Availability

ANDROVET, including its implementation and all datasets used in this study, is publicly available at <https://github.com/Demeter2025/androvet>. The repository contains the source code, usage instructions, and the results underlying the figures and tables presented in this paper.

Appendix A. Vulnerability Analysis

This appendix provides a detailed analysis of the vulnerabilities identified by ANDROVET from a user’s perspective. Table 8 presents the distribution of impact metrics across the evaluated repositories. Our goal is to shed light on their practical implications and contribute to the ongoing discussion on enhancing OS security by providing actionable insights for developers, security professionals, and end users.

To better contextualize the results of ANDROVET, we analyzed the Common Weakness Enumerations (CWEs) associated with the detected CVEs in each COS repository. For every CVE identified, we retrieved its associated CWE identifiers from the NVD, including all CWEs when multiple were assigned. Then, we aggregated the CWE counts across all repositories and normalized them by dividing by the total number of CWE occurrences, yielding a probability density function. This provides a normalized view of which weaknesses are most likely to appear in a randomly selected COS repository. Quantifying CWE frequencies in this manner offers valuable insights into the security posture of the Android-based COS ecosystem, allowing for prioritizing remediation efforts for the most prevalent and impactful weaknesses.

We focus on CWEs with a probability greater than 4%, indicating a relatively high likelihood of occurrence in a randomly selected repository. Figure 9 shows the resulting probability distribution, along with the number of COS projects affected by each CWE. The complete list of CWEs and their per-repository frequencies is available

TABLE 8. DISTRIBUTION OF VULNERABILITY IMPACT METRICS ACROSS ANDROID-BASED COS USING ANDROVET.

System	Severity				Attack Vector				User Int.		Impact									Attack		
	L	M	H	C	Ph.	Local	Net.	Adj. Net.	Yes	No	Confidentiality			Integrity			Availability			Complexity		
											N	L	H	N	L	H	N	L	H	L	M	H
ArrowOS	0	8	6	1	1	12	1	0	6	8	4	0	10	6	0	8	4	0	10	14	0	0
AOKP	0	11	15	2	1	21	4	2	14	14	4	0	24	10	0	18	8	0	20	27	0	1
CalyxOS	0	9	9	2	1	15	2	0	8	10	4	0	14	6	0	12	4	0	14	18	0	0
crDroid	0	9	10	2	1	16	2	0	9	10	4	0	15	6	0	13	4	0	15	19	0	0
dotOS	1	12	9	2	1	20	2	0	6	17	5	1	17	10	0	13	7	0	16	23	0	0
EvolutionX	0	9	10	2	1	16	2	0	9	10	4	0	15	6	0	13	4	0	15	19	0	0
GrapheneOS	0	9	10	2	1	16	2	0	8	11	3	0	16	5	0	14	4	0	15	19	0	0
LineageOS 19.0	0	9	7	1	1	14	1	0	7	9	4	0	12	7	0	9	5	0	11	16	0	0
LineageOS 20.0	0	8	7	1	1	13	1	0	7	8	4	0	11	6	0	9	4	0	11	15	0	0
LineageOS 21.0	0	9	10	1	1	16	1	0	9	9	4	0	14	6	0	12	4	0	14	18	0	0
LineageOS 22.1	0	9	10	2	1	16	2	0	9	10	4	0	15	6	0	13	4	0	15	19	0	0
Paranoid Android	0	9	10	2	1	16	2	0	9	10	4	0	15	6	0	13	4	0	15	19	0	0
Replicant	2	21	34	2	1	47	10	1	28	31	6	2	51	23	0	36	21	0	38	57	0	2
Resurrection	1	10	10	1	1	16	3	1	6	15	4	1	16	10	0	11	8	0	13	21	0	0

N = None, L = Low, M = Medium, H = High, C = Critical, Conf. = Confidentiality, Avail. = Availability, Comp. = Complexity

TABLE 9. MOST COMMON CWES OBSERVED IN ANDROID-BASED COS USING ANDROVET.

CWE Category	CWE ID(s)	Total Share (%)	Description
Memory Safety	CWE-787, CWE-119, CWE-125, CWE-416, CWE-120, CWE-191	31.17	Out-of-bounds write and buffer handling issues in low-level or native code.
Access Control/Authorization	CWE-862, CWE-284, CWE-264, CWE-863, CWE-269	28.57	Failures in access control logic, including missing or incorrect enforcement and weak permissions.
Input Validation/UI Restriction	CWE-1021, CWE-20, CWE-834	15.58	Improper input validation, especially in custom middleware or interfaces.

on our GitHub. Table 9 summarizes the most frequently observed CWEs. Several of them recur consistently across all analyzed COS repositories, most notably CWE-787 (Out-of-bounds Write), CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer), and CWE-264 (Permissions, Privileges, and Access Controls). Their persistent presence indicates systematic weaknesses in memory safety and privilege management that span multiple COS architectures and implementation layers. These issues are not isolated coding mistakes but symptoms of deeper design and architectural shortcomings. Memory-safety weaknesses are particularly concerning. They include out-of-bounds writes and reads (e.g., CWE-787, CWE-125), use-after-free (CWE-416), and classic buffer overflows (CWE-120). This category alone accounts for 31.17% of all detected weaknesses, highlighting the persistent risk posed by unsafe memory operations in low-level components such as native libraries and drivers.

Appendix B.

Source code Pre-processing.

This appendix provides a detailed description of the preprocessing pipeline used to transform raw diff data into structured representations suitable for clone detection and patch verification.

Alignment Between Diff and Target Code. Mapping diff-based change sites to real code is non-trivial due to factors such as file relocation, renaming, and partial modifications. To address this, preprocessing performs the following steps:

- 1) Reconstruct file paths using RD and RP.
- 2) Search for candidate files when exact matches fail.
- 3) Apply disambiguation based on path similarity and file type.

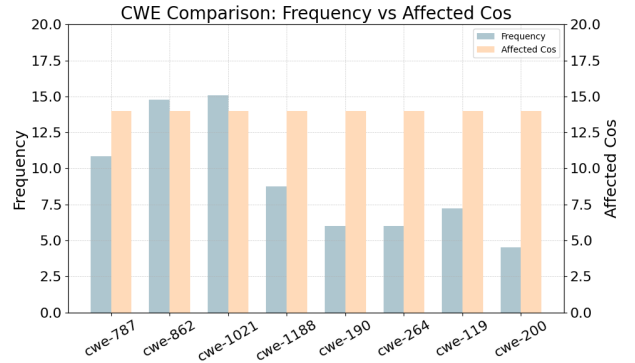


Figure 9. CWE probabilities and affected COS

This ensures that each change site is aligned with the most likely target file before analysis.

Change-Site Decomposition. Each vulnerability instance in the dataset is associated with one or more *change sites*, extracted from the corresponding diff files. A change site represents a localized modification in the source code and is decomposed into four distinct components as described in section 3.1:

- **Cnt-bfr (Context Before):** Lines immediately preceding the modification.
- **Minus (Deleted):** Lines removed from the original code.
- **Plus (Added):** Lines introduced or replacing previous ones.
- **Cnt-aft (Context After):** Lines immediately following the modification.

This decomposition follows directly from the diff structure and allows the framework to isolate both the modified code and its surrounding context.

Handling Non-Contiguous Modifications. In practice,

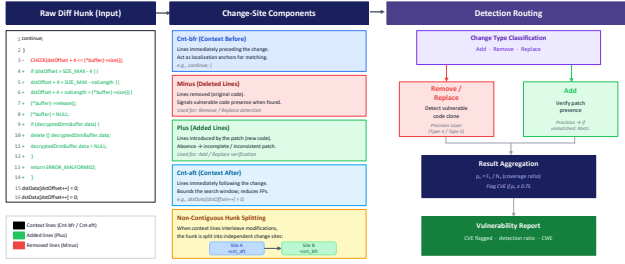


Figure 10. Workflow of the proposed system for processing change sites, including splitting and decision-making stages.

diff hunks may contain interleaved context lines between modified lines, resulting in non-contiguous change regions within a single change site. This occurs when added or removed lines are separated by unchanged code segments. To ensure that each change site represents a contiguous and semantically coherent modification, we further decompose such cases into multiple change sites. Specifically, when context lines appear between modification lines:

- The original change site is split into two (or more) independent change sites.
- The intermediate context lines are assigned as:
 - *Cnt-bfr* of the preceding change site.
 - *Cnt-aft* of the subsequent change site.
- The subsequent change site retains:
 - The remaining modification lines (*Plus* and/or *Minus*).
 - The final *Cnt-aft* lines.

This transformation converts non-contiguous diff hunks into a sequence of contiguous change sites, each with well-defined boundaries. The result is greater precision and robustness:

- **Improved localization:** Each change site corresponds to a single continuous region in the code.
- **Reduced ambiguity:** Prevents unrelated modifications from being grouped together.
- **Better matching accuracy:** Downstream matching operates on smaller, more precise units.

Without this step, interleaved context could cause misalignment during matching, leading to false positives or missed detections. Figure 10 depicts how ANDROVET processes any given change site.

Motivation for Context Splitting. The separation into *Cnt-bfr* and *Cnt-aft* is critical for three reasons:

- 1) **Localization of Matches:** Context lines act as anchors that help identify the correct region in the target codebase, especially when multiple similar code fragments exist.
- 2) **Robustness to Code Evolution:** Even when the core patch (*Plus/Minus*) is modified or partially applied, the surrounding context often remains stable, enabling approximate matching.
- 3) **Search Space Reduction:** In the abstraction layer, context lines provide structural boundaries that isolate patch-relevant instructions from surrounding code. This allows similarity computations to be confined to candidate regions, significantly reducing the search space and improving overall efficiency.

By explicitly modeling both context groups, the system avoids relying solely on modified lines, which may be insufficient in isolation.

Handling of Change Types. Each change site is categorized into one of three types:

- **Add:** Only *Plus* is present.
- **Remove:** Only *Minus* is present.
- **Replace:** Both *Minus* and *Plus* are present.

This classification directly influences downstream processing:

- *Remove/Replace* → used to detect vulnerable code. In *Replace* cases, ANDROVET will also verify patch presence.
- *Add* → used to verify patch presence.

This distinction is essential because missing patches correspond to absent “Plus” fragments close to the expected context anchors, rather than the presence of vulnerable code.